Customer Shopping Behavior Analysis

Project Overview

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences and subscription behavior to guide strategic business decisions.

Dataset Summary

- ✓ Rows: 3,900
- ✓ Columns: 18
- ✓ Key Features:
  - Customer demographics (Age, Gender, Location, Subscription Status)
  - Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
  - Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- ✓ Missing Data: 37 values in Review Rating column

Exploratory Data Analysis using Python

I began with data preparation and cleaning in Python:

- ✓ Data Loading: Imported the dataset using pandas.

```
import pandas as pd


df = pd.read_csv('customer_shopping_behavior.csv')
```

✓ Initial Exploration: Used **df.info ()** to check structure and **df.describe ()** for summary statistics.

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   str
 3   Item Purchased          3900 non-null   str
 4   Category                3900 non-null   str
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   str
 7   Size                    3900 non-null   str
 8   Color                   3900 non-null   str
 9   Season                  3900 non-null   str
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   str
 12  Shipping Type           3900 non-null   str
 13  Discount Applied        3900 non-null   str
 14  Promo Code Used         3900 non-null   str
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   str
 17  Frequency of Purchases  3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB
```

```
df.describe()
```

	Customer ID	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3900.000000	3900.000000	3900.000000	3863.000000	3900.000000
mean	1950.500000	44.068462	59.764359	3.750065	25.351538
std	1125.977353	15.207589	23.685392	0.716983	14.447125
min	1.000000	18.000000	20.000000	2.500000	1.000000
25%	975.750000	31.000000	39.000000	3.100000	13.000000
50%	1950.500000	44.000000	60.000000	3.800000	25.000000
75%	2925.250000	57.000000	81.000000	4.400000	38.000000
max	3900.000000	70.000000	100.000000	5.000000	50.000000

```
df.describe(include='all')
```

Python

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Pr Pur
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900	3900.
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2	
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No	
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223	
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN	25.
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN	14.
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN	1.
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN	13.
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN	25.
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN	38.
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN	50.

- ✓ Missing Data Handling: Checked for null values and imputed missing values in the **Review Rating** column using the median rating of each product category.

```
df.isnull().sum()
```

```
Customer ID                0
Age                        0
Gender                     0
Item Purchased             0
Category                   0
Purchase Amount (USD)      0
Location                   0
Size                       0
Color                      0
Season                     0
Review Rating             37
Subscription Status        0
Shipping Type              0
Discount Applied           0
Promo Code Used            0
Previous Purchases         0
Payment Method             0
Frequency of Purchases     0
dtype: int64
```

```
df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

Python

```
df.isnull().sum()
```

Python

```
Customer ID               0
Age                       0
Gender                    0
Item Purchased            0
Category                  0
Purchase Amount (USD)     0
Location                  0
Size                      0
Color                     0
Season                    0
Review Rating             0
Subscription Status       0
Shipping Type             0
Discount Applied          0
Promo Code Used           0
Previous Purchases        0
Payment Method            0
Frequency of Purchases    0
dtype: int64
```

- ✓ Column Standardization: Renamed columns to snake case for better readability and documentation.

```python
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df.columns
```

Python

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='str')
```

```python
df = df.rename(columns={'purchase_amount_(usd)':'purchase_amount'})
df.columns
```

Python

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='str')
```

- ✓ Feature Engineering:
  - Created age_group column by binning customer ages.

```python
# create a new column (age_group)

labels = ['Young Adult', 'Adult', 'Middle-Aged', 'Senior']

df['age_group'] = pd.qcut(df['age'], q=4, labels = labels)

df.columns
```

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases', 'age_group'],
      dtype='str')
```

```python
df[['age', 'age_group']].head(10)
```

	age	age_group
0	55	Middle-Aged
1	19	Young Adult
2	50	Middle-Aged
3	21	Young Adult
4	45	Middle-Aged
5	46	Middle-Aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-Aged

- Created purchase_frequency_days column from purchase data.

```
# Create another new column (purchase_frequency_days)

frequency_mapping = {
    'Fortnightly' : 14,
    'Weekly' : 7,
    'Monthly' : 30,
    'Quarterly' : 90,
    'Bi-Weekly' : 14,
    'Annually' : 365,
    'Every 3 Months' : 90
}
```

```
df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

```
df[['purchase_frequency_days', 'frequency_of_purchases']].head(10)
```

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

✓ Data Consistency Check: Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

```
(df['discount_applied'] == df['promo_code_used']).all() # To check if both contain the same values
```

```
np.True_
```

```
df = df.drop('promo_code_used', axis = 1) # drop one column
df.columns
```

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='str')
```

✓ Database Integration: Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis.

```python
pip install psycopg2-binary sqlalchemy
```

Python

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: psycopg2-binary in C:\Users\USER\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packag
Requirement already satisfied: sqlalchemy in C:\Users\USER\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Pyt
Requirement already satisfied: greenlet>=1 in C:\Users\USER\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Py
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\USER\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\loca
Note: you may need to restart the kernel to use updated packages.
```

```python
# step 1: connect to PostgreSQL

username = 'postgres'
password = '1234'
host = 'localhost'
port = '5432'
database = 'customer_behavior'

from sqlalchemy import create_engine

engine = create_engine(f'postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}')
```

Python

```python
# step 2: Load DataFrame into PostgreSQL

table_name = 'customer_behavior'
df.to_sql(table_name, engine, if_exists = 'replace', index = False)

print(f"Data successfully loaded into table '{table_name}', in database '{database}'.")
```

Python

```
Data successfully loaded into table 'customer_behavior', in database 'customer_behavior'.
```

Generate + Code + Markdown

```python
# Read back sample
pip install pymysql

pd.read_sql("select * from customer_behavior limit 10;", engine)
```

Python

Data Analysis using SQL (Business Transactions)

I performed structured analysis in PostgreSQL to answer key business questions:

1) Revenue by Gender - Compared total revenue generated by male vs. female customers.

*<postgres> Customer-Behaviour-Analysis.sql

```sql
--Q1. What is the total revenue generated by male vs. female customers?
select gender, sum(purchase_amount) as total_revenue from customer_behavior cb
group by cb.gender;
```

customer_behavior 1

Enter a SQL expression to filter results (use Ctrl+Space)

	gender	total_revenue
1	Female	75,191
2	Male	157,890

2) High-Spending Discount Users - Identified customers who used discounts but still spent above the average purchase amount.

*<postgres> Customer-Behaviour-Analysis.sql

```sql
--Q2. Which customers used a discount but still spent more than the average purchase amount?
select customer_id, purchase_amount from customer_behavior cb
where discount_applied = 'Yes' and cb.purchase_amount > (select avg(purchase_amount) from customer_behavior)
```

customer_behavior 1

Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	purchase_amount
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72

3) Top 5 Products by Rating - Found products with the highest average review ratings.

*<postgres> Customer-Behaviour-Analysis.sql

```sql
-- Q3. Which are the top 5 products with the highest average review rating?
select item_purchased, round(avg(review_rating::numeric), 2) as average_product_rating from customer_behavior cb
group by cb.item_purchased
order by average_product_rating desc
limit 5
```

customer_behavior 1

Enter a SQL expression to filter results (use Ctrl+Space)

	item_purchased	average_product_rating
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.8
5	Skirt	3.78

4) Shipping Type Comparison - Compared average purchase amounts between Standard and Express shipping.

```sql
--Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
select shipping_type, round(avg(purchase_amount::numeric), 2) as average_purchase_amount
from customer_behavior cb
where cb.shipping_type in ('Standard', 'Express')
group by cb.shipping_type
```

customer_behavior 1

	shipping_type	average_purchase_amount
1	Standard	58.46
2	Express	60.48

5) Subscribers vs. Non-Subscribers - Compared average spend and total revenue across subscription status

```sql
--Q5. Do subscribed customers spend more? Compare average spend and total revenue between subscribers and non-subscribers.
select subscription_status, count(customer_id) as total_customers,
round(avg(purchase_amount), 2) as average_purchase_spend,
sum(purchase_amount) as total_revenue
from customer_behavior cb
group by cb.subscription_status
order by total_revenue, average_purchase_spend desc
```

customer_behavior 1

	subscription_status	total_customers	average_purchase_spend	total_revenue
1	Yes	1,053	59.49	62,645
2	No	2,847	59.87	170,436

6) Discount-Dependent Products - Identified 5 products with the highest percentage of discounted purchases

```sql
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
select item_purchased, 100 * sum(case when discount_applied = 'Yes' then 1 else 0 end)/ count(*) as discount_percentage
from customer_behavior cb
group by cb.item_purchased
order by discount_percentage desc
limit 5
```

customer_behavior 1

	item_purchased	discount_percentage
1	Hat	50
2	Sneakers	49
3	Coat	49
4	Sweater	48
5	Pants	47

7) Customer Segmentation - Classified customers into New, Returning, and Loyal segments based on purchase history.

*<postgres> Customer-Behaviour-Analysis.sql

```sql
--Q7. Segment customers into New, Returning, and Loyal based on their total number of previous purchases, and show the count of each segment.
with customer_type as (
select customer_id, previous_purchases,
case
    when previous_purchases = 1 then 'New'
    when previous_purchases between 2 and 30 then 'Returning'
    else 'Loyal'
    end as customer_segment
from customer_behavior cb
)
select customer_segment, count (*) as Number_of_customers
from customer_type
group by customer_segment
order by number_of_customers desc
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

	customer_segment	number_of_customers
1	Returning	2,268
2	Loyal	1,549
3	New	83

8) Top 3 Products per Category - Listed the most purchased products within each category.

*<postgres> Customer-Behaviour-Analysis.sql

```sql
--Q8. What are the top 3 most purchased products within each category?
with most_purchased as (
select category, item_purchased, count(customer_id) as total_purchases,
row_number() over(partition by category order by count(customer_id) desc) as item_rank
from customer_behavior cb
group by cb.category, cb.item_purchased
)
select item_rank, category, item_purchased, total_purchases from most_purchased
where item_rank <= 3
```

customer_behavior 1

Enter a SQL expression to filter results (use Ctrl+Space)

	item_rank	category	item_purchased	total_purchases
1	1	Accessories	Jewelry	171
2	2	Accessories	Sunglasses	161
3	3	Accessories	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

9) Repeat Buyers & Subscriptions - Checked whether customers with >5 purchases are more likely to subscribe.

```sql
--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
select subscription_status, count(customer_id) as count_of_repeat_buyers from customer_behavior cb
where cb.previous_purchases > 5
group by cb.subscription_status
```

customer_behavior 1

	subscription_status	count_of_repeat_buyers
1	No	2,518
2	Yes	958

10) Revenue by Age Group - Calculated total revenue contribution of each age group

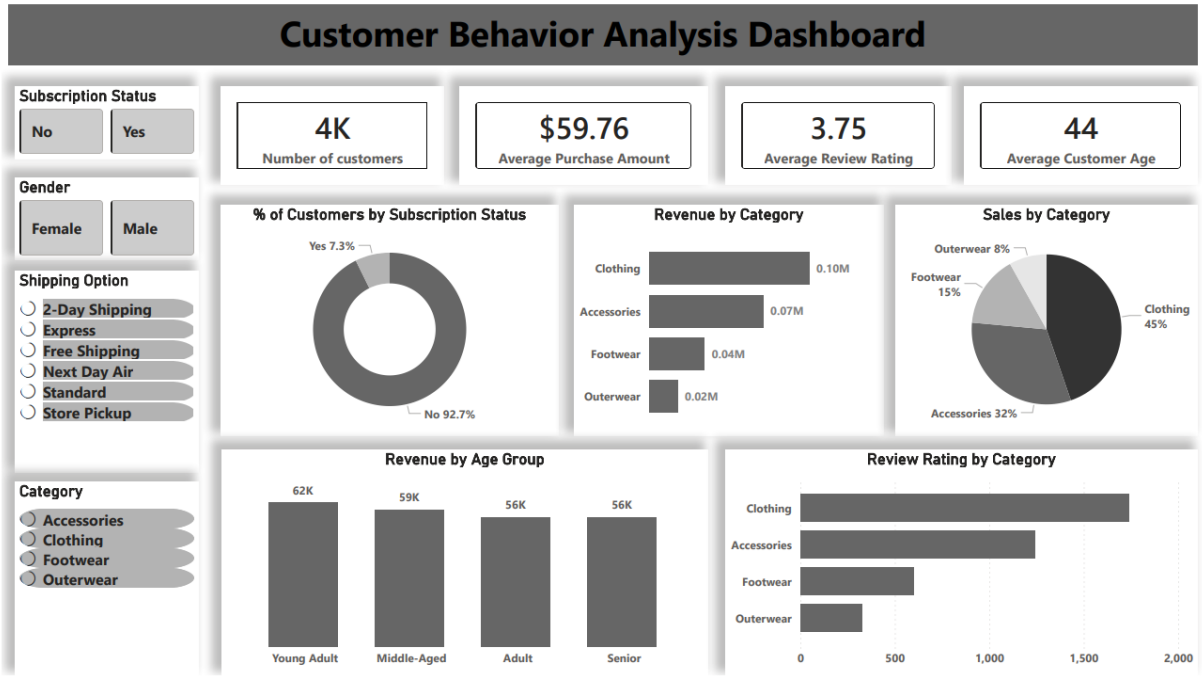
```sql
--Q10. What is the revenue contribution of each age group?
select age_group, sum(purchase_amount) as total_revenue from customer_behavior cb
group by cb.age_group
order by total_revenue desc
```

customer_behavior 1

	age_group	total_revenue
1	Young Adult	62,143
2	Middle-Aged	59,197
3	Adult	55,978
4	Senior	55,763

Dashboard in Power BI

Finally, I built an interactive dashboard in Power BI to present insights visually.



Business Recommendations

- ✓ Customer Loyalty Programs - Reward repeat buyers to move them into the “Loyal” Segment.
- ✓ Boost Subscriptions - Promote exclusive benefits for subscribers.
- ✓ Targeted Marketing - Focus efforts on high-revenue age groups and express-shipping users.

Prepared by: Peter Mutua | Data Analyst | April 2026